

/RT/ • COMPLETE PROJECT INTELLIGENCE

RepoTrajectory

Technical Project Book

A self-contained, implementation-grounded guide for engineers and AI agents: product intent, architecture, collection, storage, analytics, search, security, UI, operations, tests, risks, and safe extension points.

REPOSITORY
github_analysis

SOURCE SNAPSHOT
main @ 9e63544

GENERATED
2026-09-05 • Europe/Berlin

How to use this book

This PDF is an orientation layer over the repository, not a substitute for source control. It is written so an unfamiliar AI can form a correct mental model before editing code.

Ground-truth order

- 1. Current source code and migrations.
- 2. Executable tests and runtime behavior.
- 3. Configuration in `backend/app/core/config.py` and Compose.
- 4. This project book and checked-in Markdown docs.

Scope

Monorepo application, data pipelines, data model, API, web interface, privileged operations, analytics methodology, and local production-like deployment.

It does not embed secrets, database contents, or generated dependencies.

Evidence basis

Read from all application modules, eight migrations, tests, Docker assets, shell operations, and current live routes. Screenshots were captured from the local Compose stack.

Freshness caveat

Runtime counts are a point-in-time observation. Product logic is tied to commit `9e63544`. If HEAD differs, run `git diff 9e63544..HEAD` and revalidate affected chapters.

AI reading strategy. Read pages 3–12 for the system model, the subsystem chapter relevant to the task, then pages 53–58 for modification rules, invariants, and file ownership.

Notation

Notation	Meaning
<code>candidate</code>	Cheaply known repository; not necessarily in public directory or deeply hydrated.
<code>directory / deep</code>	Canonical v2 membership levels. Legacy collection uses <code>candidate/active/pinned</code> .
"observed"	Derived from collected GitHub/GH Archive facts, never reconstructed as ground truth.

Fifty-nine page map

01	Cover	31	Scout scoring
02	Document contract	32	AI provider degradation
03	Contents	33	Hybrid search
04	Executive summary	34	API conventions
05	Product questions and users	35	API v1 endpoint catalog
06	System context	36	API v2 endpoint catalog
07	Deployment topology	37	Admin endpoint catalog
08	Runtime request paths	38	Web application structure
09	Repository map	39	Home screenshot
10	Technology stack	40	Directory screenshot
11	Domain model and tiers	41	Scout screenshot
12	End-to-end data pipeline	42	Compare screenshot
13	Source and evidence policy	43	Repository dossier screenshot
14	GitHub Search discovery	44	Methodology screenshot
15	GH Archive processing	45	Collection screenshot
16	Classification	46	Admin screenshot
17	Reconciliation and selection	47	Security boundary
18	Deep hydration	48	Authentication and CSRF flow
19	Incremental synchronization	49	Configuration: core and collection
20	Durable queue state machine	50	Configuration: AI and capacity
21	Scheduler and job catalog	51	Local development
22	Database overview	52	Docker and proxy deployment
23	Hydrated evidence tables	53	Operations and observability
24	Discovery and operations tables	54	Backup, restore, recovery
25	Canonical catalog tables	55	Tests and quality gates
26	Analytics execution	56	Live runtime snapshot
27	Momentum score	57	Limits, risks, debt
28	Community health score	58	Safe extension playbooks
29	Concentration, velocity, confidence	59	AI handoff checklist and index
30	Directory selection score		

Executive summary

RepoTrajectory is an explainable open-source repository intelligence system. It continuously builds a bounded evidence base and presents repository discovery, health, momentum, delivery, and contributor-dependency signals through a research console.

BACKEND

FastAPI

Async SQLAlchemy and durable jobs.

FRONTEND

Next.js

Server/client UI with typed fetch layer.

STORE

PG16

Relational facts plus pgvector.

SOURCES

2

GitHub REST/Search + GH Archive.

Core idea

Separate cheap, wide discovery from expensive, narrow historical hydration. Search and GH Archive identify candidates; policy selects a cohort; workers hydrate only that bounded set; analytics store components and evidence alongside scores.

Primary outputs

- Canonical repository directory with filters and lenses.
- Hybrid lexical/semantic search.
- Scout feed for under-the-radar promise.
- Deep repository dossiers and comparison.
- Collector observability and a hardened admin plane.

Key architectural properties

- Idempotent upserts and unique job deduplication.
- Leased PostgreSQL queue safe for multiple workers.
- Bounded collection and explicit retention policies.
- Missing evidence remains missing; score weights re-normalize where defined.
- Hosted AI is optional: deterministic fallback preserves availability.
- Only Caddy exposes a host port by default.

One-sentence model: evidence sources → compact candidates → transparent selection → bounded hydration → explainable scoring → v1/v2 APIs → Next.js research workflows.

Product questions and users

Developer lens

What does this repository do? Is it active? Which technologies and topics define it? Can I discover alternatives by purpose rather than popularity?

Maintainer lens

Is delivery healthy? Are issues and pull requests resolving? Is work concentrated in too few people? What evidence is stale or weak?

Growth / investor lens

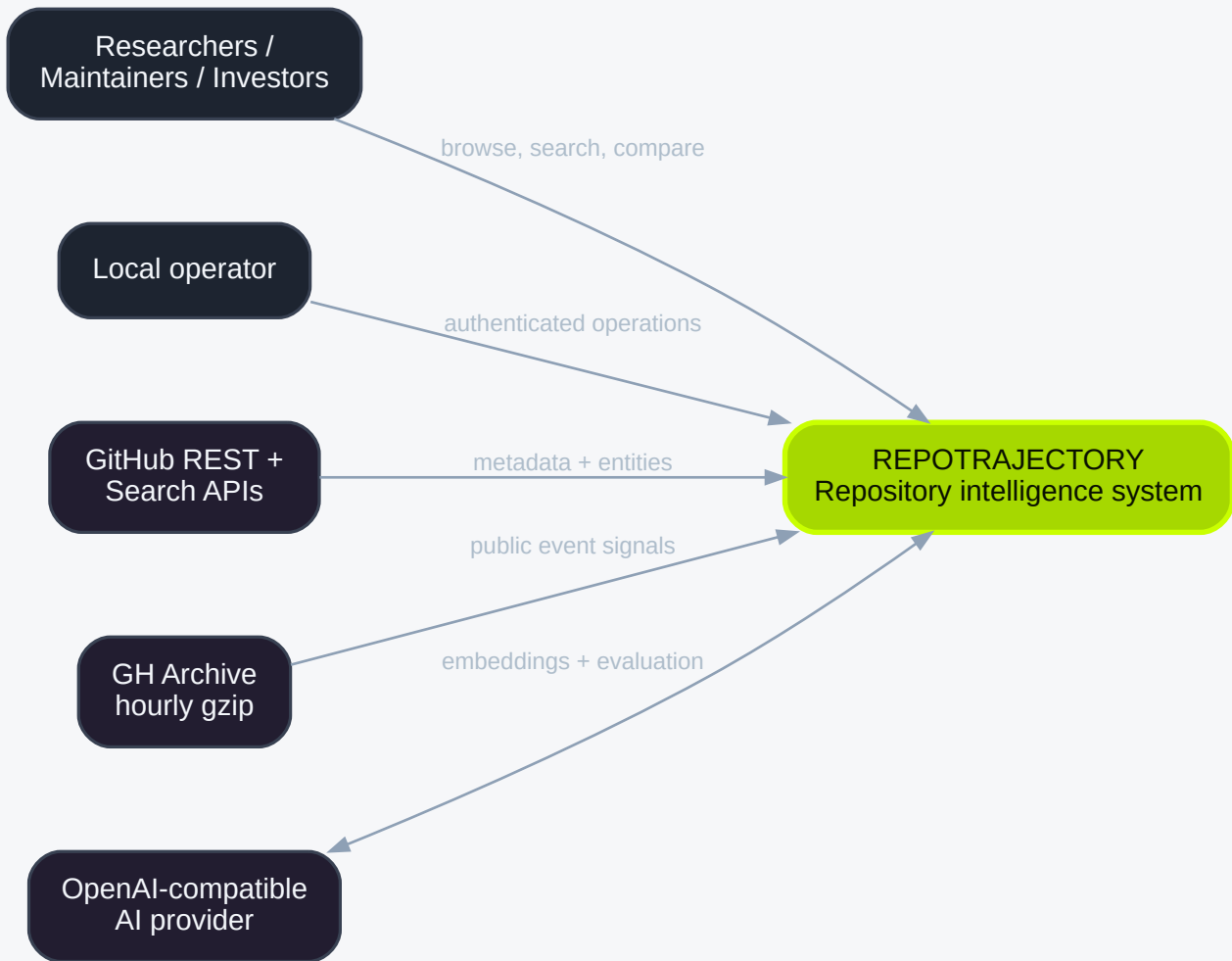
Which projects show adoption acceleration, development cadence, and early promise? What is moving, and why did the system surface it?

Research surfaces

Surface	User question	Evidence
Home	Where should I begin?	Search, analytical lenses, current Scout picks.
Repositories	What matches filters and desired viewpoint?	Catalog metadata, selection scores, deep/scout indicators.
Scout	What promising software is not already obvious?	Quantitative activity plus structured AI/fallback rationale.
Compare	How do selected projects differ?	Catalog and deep metrics, normalized presentation.
Repository dossier	What is the complete case for one project?	Profile, metrics, contributors, history, activity, provenance.
Methodology	How were scores produced?	Weights, caveats, evidence semantics.
Collection	Is the evidence pipeline current?	Queue, tiers, rate budget, archive and DB telemetry.
Admin	How do I safely intervene?	Allowlisted commands, job controls, audit ledger.

Non-goal: scores are not scientific truth, investment advice, or a literal bus factor. The product intentionally exposes components, samples, coverage, uncertainty, and freshness so humans can interpret them.

System context



Inbound evidence

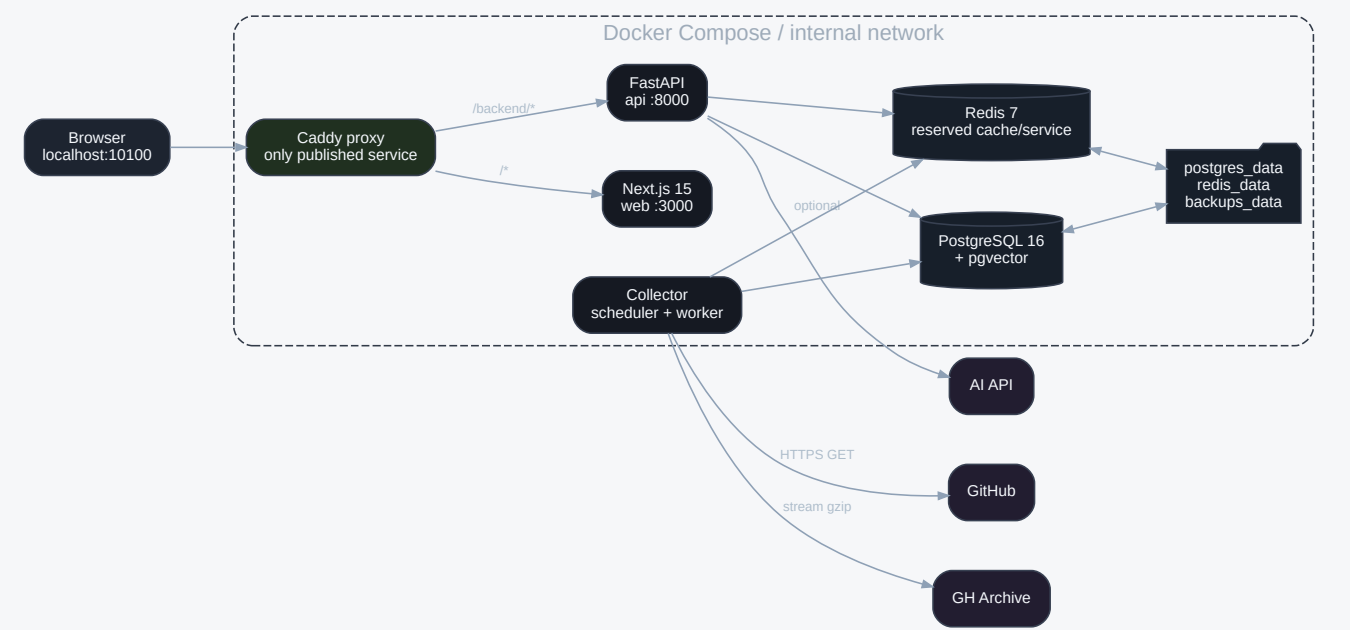
GitHub Search yields recent public software metadata. GitHub REST hydrates repository entities. GH Archive yields public event counts without retaining raw payloads. An OpenAI-compatible endpoint optionally provides embeddings and structured Scout evaluation.

Outbound behavior

Public users receive read-only research views and APIs. A local operator receives a separate authenticated command surface. RepoTrajectory issues only GET requests to GitHub and does not write back to repositories.

Trust boundary: GitHub, GH Archive, and the AI endpoint are external dependencies. Browser traffic reaches the application through Caddy; PostgreSQL and Redis remain unexposed on the host in the default Compose file.

Deployment topology



Service	Responsibility	Reachability / persistence
proxy	Caddy routes UI and <code>/backend/*</code> .	Only published port; default 10100.
web	Next.js server and client assets.	Private :3000; SSR calls <code>api:8000</code> .
api	FastAPI v1/v2/admin and OpenAPI.	Private :8000; migrates on startup.
collector	Scheduler plus one durable-job worker loop.	Private; shares backend image/config.
postgres	System of record and vector index.	<code>postgres_data</code> , <code>backups_data</code> .
redis	Provisioned infrastructure.	<code>redis_data</code> ; current source does not use it in core paths.

Important mismatch to remember: Compose sets `REDIS_URL`, but `Settings` has no Redis field and current services coordinate through PostgreSQL. Treat Redis as reserved rather than required application state.

Runtime request paths

Browser → UI/API

1. Browser requests `http://localhost:10100`.
2. Caddy sends `/backend/*` to FastAPI after stripping the prefix; all other traffic goes to Next.js.
3. Browser-side `frontend/lib/api.ts` uses `/backend`.
4. Server-rendered pages use `API_INTERNAL_URL`, defaulting to `http://localhost:8001` outside Compose.

Read query

1. FastAPI injects `AsyncSession`.
2. Route composes SQLAlchemy statements or calls a service.
3. Pydantic response models shape public contracts.
4. Security middleware adds headers globally; admin adds no-store behavior.

Background collection

1. Scheduler enqueues unique policy jobs.
2. Worker leases one due row.
3. GitHub/GH Archive calls happen outside HTTP requests.
4. Worker commits normalized evidence, analytics, catalog sync, and job outcome.

Admin mutation

1. Session cookie is decoded and expiry-checked.
2. Origin, Fetch Metadata, and CSRF are checked.
3. Allowlisted operation performs conditional or enqueue-only mutation.
4. Outcome is appended to audit log.

Public reads are synchronous • expensive ingestion is asynchronous • PostgreSQL is both evidence store and coordination plane

Repository map

Path	Purpose and ownership
backend/app/api/	HTTP contracts. <code>routes.py</code> is v1, <code>v2/routes.py</code> is canonical catalog/search/scout, <code>admin.py</code> is privileged.
backend/app/services/	Use-case logic: ingestion, discovery, collector, analytics, catalog, directory, search, Scout, AI, benchmark, admin audit.
backend/app/db/models.py	All ORM entities and relational invariants.
backend/app/analytics/metrics.py	Pure score transforms and concentration math.
backend/app/github/client.py	Rate-aware, retrying, paginated GitHub adapter.
backend/alembic/versions/	Schema history; database changes must be expressed here.
backend/tests/	SQLite-oriented unit/API tests plus provider and migration seams.
frontend/app/	Next.js App Router pages: home, repositories, dossier, Scout, compare, rankings, methodology, collection, admin.
frontend/components/	Interactive clients and reusable visual primitives.
frontend/lib/api.ts	Browser/server base selection, TypeScript API types, request helpers.
docs/	Collector, metrics, administration notes and generated project book.
run.sh / Makefile	Developer/operations entry points.
docker-compose*.yaml / Caddyfile	Runtime topology and local proxy contract.
analysis/	Decoupled research workspace; not on the application runtime path.

Technology stack and dependency posture

Backend

Python	≥3.12
FastAPI	≥0.115,<1
SQLAlchemy	async 2.x
PostgreSQL driver	asyncpg
Migrations	Alembic
HTTP client	httpx
Vector type	pgvector
CLI	Typer
Logging	structlog
Serialization	Pydantic + orjson dependency

Frontend

Next.js	^16.3.1
React / React DOM	^19
TypeScript	^5.7
Tailwind	^3.4
Motion	^13.1.1
Icons	Heroicons 2
Charts	Native SVG components

Infrastructure

pgvector/pgvector:pg16	redis:7-alpine	caddy:2-alpine	Docker Compose	named volumes
------------------------	----------------	----------------	----------------	---------------

Quality posture

Backend declares strict MyPy, Ruff rules **E, F, I, UP, B, ASYNC**, pytest-asyncio, and respx. The frontend's **build** is the effective type/build gate. Dependency ranges intentionally stay within major versions; lock files are present for both Python (**uv.lock**) and npm.

Version note. The current package declares Next.js 16.3.1 even if older prose says Next.js generically. For upgrade work, trust **frontend/package.json** and its lockfile.

Domain model and membership tiers

Two related models coexist: the original collection/hydration model and the newer canonical catalog model. They are linked but must not be conflated.

Legacy collection track

`RepositoryCandidate` is a discovered row. Tier values:

- **candidate**: known, not selected.
- **active**: selected by collection policy for recurring hydration.
- **pinned**: explicitly requested, highest priority.

`Repository` is the deeply hydrated normalized record.

Canonical catalog track (v2)

`CatalogRepository` unifies broad discovery and hydrated facts. Flags/tiers:

- **candidate**: retained pool.
- **directory**: selected public catalog.
- **deep**: highest-value directory subset, preferably hydrated.

Scout, embeddings, provenance, and search documents attach here.

Stable identity

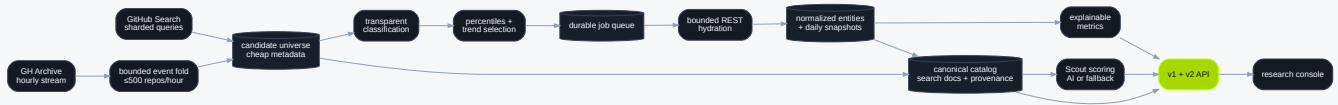
GitHub numeric `github_id` is the durable external identity; `full_name` can change on rename/transfer. The GitHub client follows redirects, and catalog/repository upserts store the canonical response identity.

Key linkage

```
RepositoryCandidate.repository_id — optional 1:1 → Repository.id
CatalogRepository.repository_id — optional 1:1 → Repository.id
CatalogRepository.github_id      — parent key → search / embedding / scout
```

Implementation caution. Some services are mature legacy collection paths and others are v2 canonical paths. A new feature should declare which universe it reads and how synchronization is maintained.

End-to-end evidence pipeline



Wide and cheap

Discovery is designed for tens of thousands of repositories. Metadata and compressed hourly counters are sufficient to classify and prioritize without paying deep REST cost.

Narrow and expensive

Hydration is bounded by time windows and item limits. Only selected or pinned repositories receive commits, PRs, issues, releases, contributors, snapshots, and metrics.

Essential invariant: candidate selection can be replayed; deep evidence is incremental and idempotent; raw GH Archive event JSON is never persisted.

Source and evidence policy

Source	What is collected	What is not assumed
GitHub Search	ID, owner/name, description, language, topics, stars/forks, pushed time, fork/archive flags.	Search order is not a quality ranking; coverage depends on query shards and API caps.
GH Archive	Hourly counts of Watch, Fork, Push, PullRequest, Issues, and Release events per repository.	No raw payload retention; counters do not equal unique humans or complete repository history.
GitHub REST	Canonical repo metadata, default-branch commits, recent issues/PRs/releases, contributor aggregates, languages/topics, README.	No CI reliability, review detail, permissions, tacit maintainer knowledge, or historical stars before observation.
Hosted AI	Text embedding and schema-constrained Scout judgment.	It never becomes sole evidence; service can degrade to deterministic fallback.

Storage bounds

RAW ARCHIVE JSON

0

Never stored.

EXTERNAL ACTIVITY

90d

Default retention.

README

20K

Characters bounded; 500-character excerpt.

Epistemic rule: “missing” is not “neutral.” Growth components are excluded until snapshot coverage exists; other missing response evidence can score zero and remains visible with sample sizes.

GitHub Search discovery

Legacy daily language search

`services/discovery.py` builds per-language searches with default eligibility of at least 500 stars and a push within 180 days. Results become `RepositoryCandidate` rows and collection memberships.

Canonical sharded search

`services/directory.py` searches language × star-band × freshness shards, allowing low-star projects into the v2 pool. Default scheduled shards cover the first two configured languages and bands 0–10, 11–50, and 51–200.

Canonical upsert behavior

- Conflict key: numeric GitHub ID.
- Initial classification is `software` with 0.8 confidence for the sharded path.
- Normalized topics are lowercased and underscores become hyphens.
- Search document is inserted/updated in the same use case.
- Observation timestamps distinguish discovery from last observation.

Search limits

GitHub Search has its own smaller rate bucket and result ceilings. The client preserves a configured reserve and scheduled queries use dedupe keys scoped by language and day.

```
language:{language} stars:{min}..{max} pushed:>={cutoff} fork:false
```

```
Default languages: Python, TypeScript, JavaScript, Go, Rust, Java
```

Why two paths? The legacy collection prioritizes established active software for deep analytics; canonical sharding broadens the directory and Scout funnel toward emerging projects.

GH Archive streaming and projection

Each lagged hourly `.json.gz` object is streamed, parsed line-by-line, folded into bounded counters, and discarded.

Recognized events

- `WatchEvent` → star
- `ForkEvent` → fork
- `PushEvent` → push
- `PullRequestEvent` → collaboration
- `IssuesEvent` → collaboration
- `ReleaseEvent` → delivery

Selection behavior

Adoption and collaboration lead ranking. Push volume is capped as supporting evidence to prevent noisy push streams from dominating. At most 500 repositories per hour are retained by default.

Projection lifecycle

1. Scheduler chooses six hours ending three hours behind wall clock.
2. `GhArchiveClient.read_hour` streams compressed bytes and events.
3. Projector returns counters plus event and byte totals.
4. `persist_gh_archive_hour` maps external repositories to candidates and atomically replaces that hour's rows.
5. `GhArchiveFile` records status, counts, bytes, algorithm version, and errors.

Versioning invariant. Dedupe keys and file records contain `GH_ARCHIVE_ALGORITHM_VERSION`. A formula change must bump the version so the configured hours replay and old projections do not mix with new ones.

Primary code: `backend/app/services/discovery.py` • orchestration: `backend/app/services/collector.py`

Transparent repository classification

Classification gates expensive hydration and makes rejection explainable. It uses repository metadata, not an opaque model.

Class family	Examples of signals	Hydration
<code>library</code> , <code>framework</code> , <code>developer_tool</code> , <code>software</code>	Package/framework/tool/app terms, software topics, focused descriptions.	Eligible when other guards pass.
<code>learning_resource</code>	Awesome lists, tutorials, interview material, courses, roadmaps.	Withheld.
<code>template</code>	Boilerplate, starter, template signals.	Withheld.
<code>unclassified</code>	Insufficient metadata or ambiguity.	May receive one-request probe, bounded per reconcile.

Decision outputs

`RepositoryClassification` carries the label, confidence, eligibility, and a human-readable rejection reason. Candidate rows persist each field so collection UIs and operators can inspect policy outcomes.

Probe path

Uncertain GH Archive discoveries often lack rich metadata. The reconciler returns a bounded list of candidate IDs; `probe_repository` fetches one repository record, reapplies classification, then queues another reconcile.

Known limitation: this is an intake heuristic, not semantic truth. False positives/negatives are expected. Manual pinning exists for edge cases, and reclassification is an explicit CLI/admin operation.

Reconciliation and cohort selection

Legacy active cohort

`reconcile_collection` combines candidate metadata with recent external activity. Seven-day signal percentiles and popularity inform `trend_score`. Eligible software is ranked; up to `collector_active_limit` (250 default) becomes active. Pinned entries remain privileged.

Hydration consequence

New active candidates enqueue `hydrate_repository`. Existing active repositories become due after their next refresh. Queue priorities are recomputed from tier and trend score.

Canonical directory

`reconcile_directory_and_cohort` selects up to 10,000 eligible catalog rows by selection score, stars, and recency while capping any language at 25%. Empty slots are filled by the next-best entries.

Deep subset and pruning

Up to 500 directory rows become deep, preferring already-hydrated repositories. Nonmembers older than 90 days are pruned, and the total pool is capped at 50,000 by deleting lowest-score nonmembers.

```
legacy priority: pinned = 100
active hydration = 25 + min(70, round(trend_score))

canonical order: selection_score DESC, stars DESC, pushed_at DESC
deep preference: hydrated, then selection_score, then stars
```

Transactional intent: flags and memberships describe the current policy result; job deduplication prevents each reconcile from creating duplicate work for the same time bucket.

Deep GitHub REST hydration

RepositoryIngester turns one **owner/repo** into normalized evidence with conflict-safe updates and strict safety ceilings.

Resource	Default bound	Persistence
Repository metadata	One canonical response	repositories ; follows renames.
Commits	180 bootstrap days; 5,000 items	Default branch, SHA unique per repository.
Issues	3,000 recently updated	Pull-request-shaped issue payloads excluded.
Pull requests	3,000 recently updated	Created/closed/merged state and authors.
Releases	730 days; 500 items	Draft/prerelease flags retained.
Contributors	200	Contributor identity plus cumulative counts.
Languages/topics	GitHub response	Replacement tables per repository.
README	20,000 characters downstream	Search document + 500-char catalog excerpt.

After ingestion

A UTC-day repository snapshot is upserted, metrics are recalculated for 30 days, and the hydrated repository is synchronized into the canonical catalog/search document. The candidate receives the resulting repository link and a next refresh timestamp.

Interpretation: ceilings define a recent bounded view, not completeness. A ceiling hit must be treated as coverage metadata, especially for very large repositories.

Incremental synchronization and GitHub behavior

Per-resource state

`repository_sync_states` stores `resource`, time watermark, ETag, Last-Modified, last success/error, and next sync. Subsequent collection uses watermarks to stop pagination once known history is reached.

Idempotency

Repositories, commits, issues, PRs, releases, contributors, languages, and topics use conflict-aware inserts/updates or replacement semantics. A UTC-day snapshot is updated rather than duplicated.

Rate-budget behavior

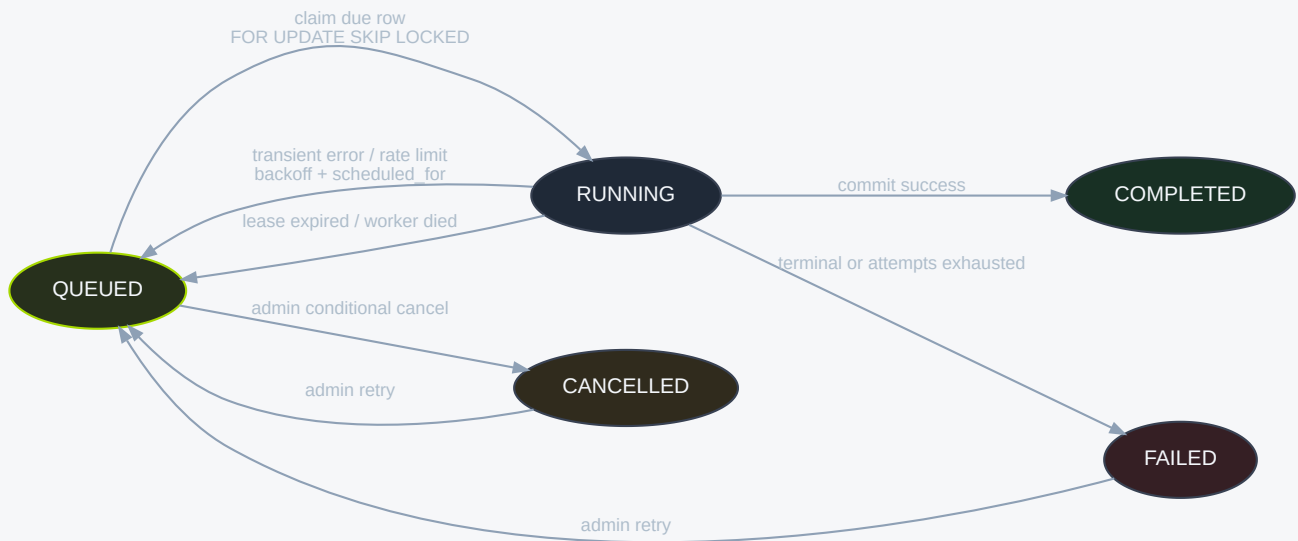
The client tracks `limit`, `remaining`, `used`, `reset_at`, `resource`, and per-process request count. The worker persists it in `collector_state`. When the reserve is reached or GitHub returns exhaustion, the current job is rescheduled after reset rather than failed.

Client mechanics

- 30-second HTTP timeout and up to three retries.
- Serial request lock per worker client.
- Minimum request interval, default 0.15s.
- Exponential retry for 429/5xx and secondary limits.
- Primary rate reset is parsed and surfaced as a typed exception.
- Redirects followed for rename/transfer.
- Pagination follows Link headers and supports stop conditions.

Scaling rule: additional workers improve parsing and database concurrency but cannot create GitHub quota. Observe budget and DB latency before scaling.

Durable queue state machine



Claim protocol

A worker selects one due queued row ordered by priority, schedule, then ID with **FOR UPDATE SKIP LOCKED**. It writes worker ID, lock time, first start time, and increments attempts before execution.

Recovery protocol

Every scheduler tick requeues running jobs whose lease exceeds the configured 30-minute default. Transient failures receive backoff; terminal/exhausted jobs remain visible.

Concurrency protection: admin retry/cancel uses status-conditional updates so a UI action cannot overwrite a job claimed concurrently by a worker.

Scheduler and job catalog

Job type	Default priority	Effect
discover_github	100	Daily configured-language legacy search.
discover_gharchive	90	Stream and persist one versioned hour.
reconcile_directory	85	Rebuild canonical directory/deep memberships and prune.
scout_eval_batch	80	Evaluate expired/unevaluated candidates.
generate_embeddings	70	Generate missing/outdated catalog vectors.
discover_sharded	60	Low-star canonical search shards.
reconcile_collection	50	Classify/rank legacy candidates; emit hydrate/probe jobs.
probe_repository	45	One-request metadata clarification.
hydrate_repository/refresh_repository	25–95	Deep evidence, metrics, catalog sync.
ingest_repository	100 when pinned	Explicit one-off ingestion by full name.
maintenance	–10	Delete expired GH Archive projections/file ledgers.

Time-bucketed `dedupe_key` values make repeated scheduler ticks safe. On cancellation, the same key can be reactivated with cleared attempts/locks. The continuous collector alternates scheduling and claiming with a default 10-second polling interval.

Database overview



This is a subsystem ER view, not every column. Cascade behavior is concentrated around repository/candidate/catalog children. Numeric GitHub ID connects external identity across broad and deep models.

System of record: PostgreSQL contains facts, derived snapshots, queue state, audit state, and vectors. Redis is not the authority for any current behavior.

Hydrated evidence tables

Table / ORM	Key fields	Role and constraints
<code>repositories</code>	ID, github_id, owner/name, metadata, last_ingested	Deep canonical GitHub record; external IDs and full name indexed/unique.
<code>repository_snapshots</code>	repository, captured_at, stars/forks/issues/watchers/contributors	Observed growth baseline; one updated point per UTC day.
<code>contributors</code>	github_id, login, avatar	Person/account identity.
<code>repository_contributors</code>	repository + contributor, contributions	Many-to-many cumulative GitHub contribution count.
<code>commits</code>	repository + SHA, author, committed_at	Default-branch commit evidence; repo-scoped identity fixed in migration 0002.
<code>pull_requests</code>	repository + number, state, created/closed/merged	Cycle time and acceptance evidence.
<code>issues</code>	repository + number, state, created/closed	Issue intake and close-time evidence, excluding PR payloads.
<code>releases</code>	repository + github_id, publish time, draft/prerelease	Stable release cadence.
<code>repository_languages</code>	repository + language, bytes	Language footprint.
<code>repository_topics</code>	repository + topic	Normalized GitHub topics.
<code>metric_snapshots</code>	repo, time, window, 3 scores, JSON components	Append-only derived assessment and full explanation payload.

Definition: backend/app/db/models.py lines 25-191 • creation evolved from Alembic 0001/0002/0003.

Discovery and operations tables

Table	Purpose	Important invariant
<code>collections</code>	Named policy: candidate/active limits, refresh interval, enabled.	Stable unique slug.
<code>repository_candidates</code>	Cheap metadata, classification, trend components, tier, refresh date.	Unique github_id, full_name, and optional repository link.
<code>collection_memberships</code>	Candidate rank, score, selection within a collection.	Composite collection + candidate key.
<code>ingestion_jobs</code>	Type/status/priority/schedule, lease, attempts, payload, error.	Unique dedupe key; explicit lifecycle timestamps.
<code>repository_sync_states</code>	Incremental watermarks and conditional headers by resource.	Composite repository + resource key.
<code>external_repository_activity</code>	Compact hourly event counters and weighted total.	Composite candidate + hour; retained 90 days by default.
<code>gh_archive_files</code>	Hour status, projection version, event/repo/byte counts, errors.	One ledger row per archive hour.
<code>collector_state</code>	Small JSON operational facts such as GitHub rate budget.	String key upsert.
<code>admin_audit_log</code>	Actor/action/target/outcome/remote address/details.	Append-only by application convention; excludes secrets.

Do not “clean up” old job rows casually. The queue is also an operational ledger. Migration 0008 removes duplicate schedule/maintenance jobs and adds a partial unique index for active equivalents.

Canonical catalog, search, and Scout tables

Table	Payload	Lifecycle
catalog_repositories	Identity, metadata, membership flags, classification, component/selection scores, Scout pointer fields, excerpt, provenance JSON, deep link.	Upserted from broad discovery and deep repository sync.
repository_search_documents	Name, owner, description, topics text, language/license, README text.	One row per catalog repository; lexical search source.
repository_signal_snapshots	Stars/forks/issues/watchers, velocity score, source.	Canonical observed time series; separate from legacy snapshots.
repository_embeddings	Version, model, vector(1536), timestamps.	Regenerated when content/model/version changes.
scout_assessments	Promise, quantitative and AI scores, confidence, rationale, facts, risk flags, evidence refs, provider identity, expiry/current flag.	Versioned history; one current logical assessment per repo by service convention.
repository_provenance	Field, source, observed time, details.	Evidence lineage records.
query_embedding_cache	Normalized query + model → JSON vector.	Avoids repeated provider calls; composite key.

Vector dimensionality is schema-bound. The model column is `Vector(1536)`. Changing `AI_EMBEDDING_DIMENSION` alone is insufficient; update migration, model, provider expectations, stored rows, and search tests together.

Analytics execution and stored explanation

`calculate_metrics(session, repo, window_days=30)` compares a current window with the immediately preceding equal window, then appends a `MetricSnapshot`.

Queries performed

- Human vs known-bot commit authors.
- PR creation and resolution/merge timestamps.
- Issue creation and close timestamps.
- Stable releases only.
- Recent contributors or all-time fallback.
- Repository snapshot history and freshness maxima.

Stored component groups

- `methodology_version: 3`
- `momentum, health, concentration`
- `velocity, responsiveness, community`
- `freshness, growth`
- `assessment, data_quality`

Bot heuristic

Logins ending in `[bot]` and the explicit values `dependabot`, `dependabot-preview`, `github-actions`, and `pre-commit-ci` are automation. Bot commits are excluded from human momentum/health inputs but stored as volume and share.

Assessment status

Archived	Repository flag is archived.
Dormant	No commits/PRs and last commit absent or >90 days.
Cooling	Momentum <45 or commit change <-25%.
Advancing	Momentum and health both ≥65.
Stable	All other cases.

Momentum score

Question: is development and community activity accelerating? Every component is clamped to 0–100.

Component	Transform	Weight
Star growth	$50 + 50 \cdot \tanh(\text{star_rate} \cdot 4)$	25%
Contributor growth	$50 + 50 \cdot \tanh(\text{contributor_rate} \cdot 3)$	20%
Commit acceleration	$50 + 50 \cdot \tanh(\text{ratio_change}(\text{current}, \text{previous}))$	25%
PR acceleration	Same acceleration transform	20%
Release cadence	$100 \cdot (1 - \exp(-\text{releases_per_month}/2))$	10%

Ratio convention

```
ratio_change(current, previous):
  previous > 0 → (current - previous) / previous
  previous = 0 and current > 0 → +1.0
  both zero → 0.0
```

Baseline handling

Star and contributor growth use an actual snapshot at or before the start of the window. If none exists, both components are excluded and remaining weights re-normalize. If there is also no current commit, PR, or release evidence, momentum is explicitly forced to 0.

Do not reconstruct historical stars. GitHub REST returns current counts. Growth is valid only from observations collected over time.

Community health score

Question: is the community active, responsive, and able to ship? Scale features use diminishing returns so raw project size does not dominate.

Component	Transform	Weight
Active human contributors	$100 \cdot (1 - \exp(-\text{contributors}/12))$	20%
Issue resolution	$100 \cdot \exp(-\text{median_close_hours}/(72 \cdot 4))$; missing $\rightarrow 0$	15%
PR merge time	$100 \cdot \exp(-\text{median_merge_hours}/(48 \cdot 4))$; missing $\rightarrow 0$	15%
PR merge rate	Merged / resolved PRs $\times 100$	20%
Stable release cadence	$100 \cdot (1 - \exp(-\text{releases_per_month}/2))$	10%
Recent human commits	$100 \cdot (1 - \exp(-\text{commits_per_week}/15))$	20%

Cycle-time cohorts are selected by *resolved during the window*, reducing recent-open cohort bias. Issue close time is not first-response time; PR merge time is not review latency. Acceptance rate can encode deliberate governance strictness, not only health.

Evidence visibility: the snapshot stores median hours plus merged, resolved-PR, and closed-issue sample sizes. A consumer should show those counts next to the score.

Concentration, velocity, and confidence

Contributor-dependency proxy

```
top_1_share = largest contributor count / total
top_3_share = three largest counts / total
HHI =  $\sum(\text{each contributor share}^2)$ 
risk =  $100 \times (0.45 \cdot \text{top\_1} + 0.25 \cdot \text{top\_3} + 0.30 \cdot \text{HHI})$ 
effective_contributors =  $1 / \text{HHI}$ 
```

Basis is recent human commits. If none exist, cumulative GitHub contribution counts are used and the basis is marked. This is not a literal bus factor: it does not know permissions, knowledge ownership, or succession.

Velocity payload

Current/previous commits and PRs, merged PRs, issues opened/closed, net issue flow, stable releases, per-week/month normalized rates, and automated commit volume/share.

Freshness payload

Last commit/release timestamps and days since each. Repository dossier and assessment use these to avoid presenting stale evidence as current.

Confidence score

```
min(100,
  25
  min(current evidence events, 150) / 150 × 50
  min(snapshot span days, window days) / window days × 25)
```

The 25-point base means “confidence” is a coarse coverage indicator, not statistical certainty. Sample sizes remain separate.

Directory selection score

`services/directory.py` computes a 0–100 score from broad evidence. The score selects directory membership; it is not the same as deep health or Scout promise.

Component	Formula / behavior	Weight
Activity	<code>min(100, 20 + activity_count·2.5)</code>	35%
Popularity	<code>min(100, log1p(stars)·8 + log1p(forks)·4)</code>	30%
Freshness	<code>max(0, 100 - days_since_push)</code> ; missing defaults 20	20%
Maintenance metadata	50 base +20 license +15 meaningful description +15 at least two topics	15%

The catalog row also stores individual `activity_score`, `popularity_score`, `freshness_score`, and `maintenance_score` fields. Selection then applies a language diversity cap to prevent one ecosystem from consuming more than 25% of target slots, followed by a fill pass.

Current implementation nuance: some discovery upserts seed component/selection scores with constants or simplified values, while deep sync seeds stronger values. If revising selection quality, trace every writer of `CatalogRepository.selection_score`, not only `calculate_selection_score`.

Scout promise scoring

Scout seeks promising active software with no minimum star threshold. It combines deterministic evidence (70%) and structured provider evaluation (30%).

Quantitative component	Construction
Adoption acceleration	20 base + 15/star event + 10/fork event + bounded log-star term.
Development cadence	30 base + 3/push event + 5/PR event.
Freshness	100 – 1.5 per day since push; missing 20.
Contributor breadth	25 base + forks, PR events, issue events.
Release activity	20 base + 35/release event.
Maintenance responsiveness	50 base; deep merged/closed evidence can raise it.

```
quantitative = .25 adoption + .20 cadence + .15 freshness
              + .15 breadth + .10 releases + .15 maintenance
promise      = .70 quantitative + .30 provider_overall_score
```

Separate confidence accumulates evidence presence: description .20, language .15, license .15, ≥2 topics .15, any recent events .20, ≥5 stars .15; bounded to 0.20–1.00.

Guardrails exclude forks, archived projects, known non-software classifications, and projects inactive for more than 180 days. Assessments expire after seven days and retain provider/prompt identity plus evidence references.

AI provider contract and graceful degradation

Abstract interface

`AIProvider` exposes embedding generation, structured Scout evaluation, availability, and model identity.

`ScoutAIEvaluation` requires an overall score, breakdown, rationale, surfaced reason, facts, uncertainty, risks, model, and prompt version.

Hosted provider

`OpenAIProvider` uses the OpenAI-compatible base URL, bearer key, embedding endpoint, and chat-completions JSON response. Default base URL targets Google's OpenAI-compatible Gemini endpoint.

Fallback provider

When no effective key exists or provider access fails in supported paths, deterministic heuristics generate embeddings/evaluation behavior. The v2 health route reports degraded status and `semantic_available` tells search consumers what ran.

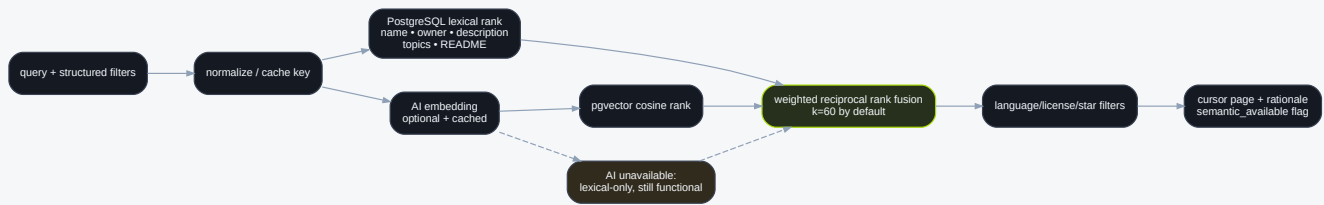
Factory

`get_ai_provider` selects hosted behavior only when `GEMINI_API_KEY` or `AI_API_KEY` is configured; Gemini key wins.

Design consequence: the system remains usable without hosted AI. Lexical search, broad catalog, deep metrics, and deterministic Scout evidence continue. Never make core read availability depend on provider credentials.

Naming nuance: class/file names say OpenAI because the protocol is OpenAI-compatible; the configured default evaluation model is `gemini-3.8-flash`.

Hybrid search and reciprocal rank fusion



Lexical branch

Searches normalized catalog text across full name, name, owner, description, topics, language/license, and README document fields. Structured filters are applied in SQL.

Semantic branch

Normalizes query, consults `query_embedding_cache`, requests an embedding if needed, and ranks catalog vectors by pgvector distance.

Rank lists combine through weighted reciprocal rank fusion with default `k=60`. Pagination uses a URL-safe base64 encoding of an offset. Responses include interpreted filters, rationale, evidence freshness, and an explicit semantic-availability flag.

Cursor semantics: the cursor is opaque to clients but currently offset-based, not keyset-based. Large/mutating result sets can shift between pages; a future keyset migration would be an API behavior change.

API architecture and conventions

Mounts

- `/health`: process liveness.
- `/api/v1`: legacy repositories, analytics, collection, admin.
- `/api/v2`: canonical directory, profile, search, Scout, facets, health.
- Via Caddy, prepend `/backend`.

Session and errors

Async session dependency is request-scoped. Missing repositories produce 404; invalid parameters produce FastAPI 422. Most read responses are Pydantic-shaped. Admin failures use 401/403/409 as appropriate.

Conventions

- All timestamps are timezone-aware ISO 8601 externally.
- Ranking and score ranges are 0–100; Scout confidence is 0–1.
- v2 lists return envelopes with `items`, `next_cursor`, and `total_count`.
- Frontend requests use `cache: "no-store"`.
- OpenAPI metadata advertises API version 0.2.0; Python package version is 0.1.0.
- Global response headers deny framing and sniffing, restrict referrers/permissions, and constrain CSP frame/base behavior.

Client base resolution

```
browser:      /backend
SSR in Compose: API_INTERNAL_URL=http://api:8000
SSR local:    http://localhost:8001
```

Compatibility guidance: keep v1 behavior stable for existing screens while adding canonical functionality to v2. If a type changes, update Pydantic schemas and `frontend/lib/api.ts` in the same change.

API v1 endpoint catalog

Method and path	Purpose / key parameters
GET /repositories	Hydrated rows; search, tier, offset, limit≤500.
GET /repositories/{owner}/{repo}	One hydrated summary.
GET .../metrics	Latest score for window 7–365; optional refresh recalculates.
GET .../history	Observed snapshots; days≤3650.
GET .../contributors	Top contributor rows; limit≤100.
GET .../activity	Weekly commits, merged PRs, closed issues, releases; 1–104 weeks.
GET /rankings/{ranking}	momentum, health, or concentration order; window and limit.
GET /collections	Collection policy and member counts.
GET /candidates	Filter legacy candidates by tier/language/class/search/eligible.
GET /trending	Highest legacy trend candidates.
GET /collector/overview	Tier/class/job/rate/archive/DB summary.
GET /collector/jobs	Recent job ledger, optional status.
POST /collector/schedule	Legacy mutation route; runs scheduling.
POST /collector/repositories	Queue/pin an owner/repo.
POST /collector/jobs/{id}/retry	Retry eligible job.

Security review point: v1 exposes legacy collector mutation routes outside the /admin router. Before public deployment, verify their current dependencies and either protect or remove them; do not assume the admin chapter automatically covers these routes.

API v2 endpoint catalog

Method and path	Contract
GET /repositories	Canonical directory envelope. Filters: language, license, tier, directory_only, star range. Lenses: developer/maintainer/investor. Sort: selection, stars, pushed, promise, activity, growth, health, name; order asc/desc; cursor; limit 1–100.
GET /repositories/{owner}/{repo}	Unified profile: catalog row, current Scout card, optional deep evidence (metric, contributors, history), provenance, README excerpt.
POST /search	Body: query, optional filters/cursor/limit. Returns fused items, interpreted filters, rationale, freshness, semantic status.
GET /scout	Current Scout feed with language, minimum promise, maximum stars, cursor, and limit controls.
GET /facets	Language, category, license facets plus evidence-level and freshness counts.
GET /health	Database and AI availability plus candidate/directory/deep counts and degraded features.

Lens metrics

List rows include a computed `lens_metrics` object. The UI uses it to rename and reorder evidence for developer, maintainer, or investor/growth perspectives without duplicating the underlying catalog.

Canonical read choice: new repository discovery/profile features should usually start with v2. Use v1 only when deep historical analytics or legacy collector observability is the explicit need.

Administration endpoint catalog

Method and path	Authorization	Effect
POST /admin/auth/login	Credentials + rate limiter	Set signed cookie; return CSRF and expiry.
GET /admin/auth/session	Session cookie	Return current session details.
POST /admin/auth/logout	Full mutation checks	Clear cookie; audit sign-out.
GET /admin/summary	Session cookie	Detailed operations summary.
GET /admin/audit	Session cookie	Latest append-only audit records, limits≤500.
POST /admin/commands/{command}	Session + Origin + Fetch Metadata + CSRF	Allowlisted scheduler, reconcile, reclassify, or maintenance enqueue.
POST /admin/jobs/{id}/cancel	Full mutation checks	Conditional cancel of queued/failed job.

Allowlisted command names

- schedule
- reconcile
- reclassify
- maintenance

Repository queueing and retry behavior are surfaced in the admin console through protected calls where implemented; no endpoint exposes shell, SQL, arbitrary job type, Docker socket, or raw environment data.

Long-running guarantee: HTTP commands enqueue work. They do not perform GitHub collection inside the request, preserving queue leases, retry policy, and rate-limit behavior.

Web application structure

Route	Rendering / primary component	Responsibility
/	Async server page + cards	Search entry, lenses, Scout preview, platform explanation.
/repositories	Server page → <code>RepositoryDirectory</code>	Faceted canonical catalog, lens/sort/search controls.
/repositories/[owner]/[repo]	Async server dossier	Unified profile with deep/scout/evidence-aware sections.
/scout	Server shell → <code>ScoutFeed</code>	Promise filters and expandable explanations.
/compare	Server seed → <code>CompareExplorer</code>	Select and compare repositories.
/rankings	Server seed → <code>RankingWorkspace</code>	Legacy metric ranking workspace.
/methodology	Static server content	Metric and evidence governance.
/collection	<code>CollectorConsole</code>	Read-only queue/source observability.
/admin	<code>AdminConsole</code>	Login and privileged controls.

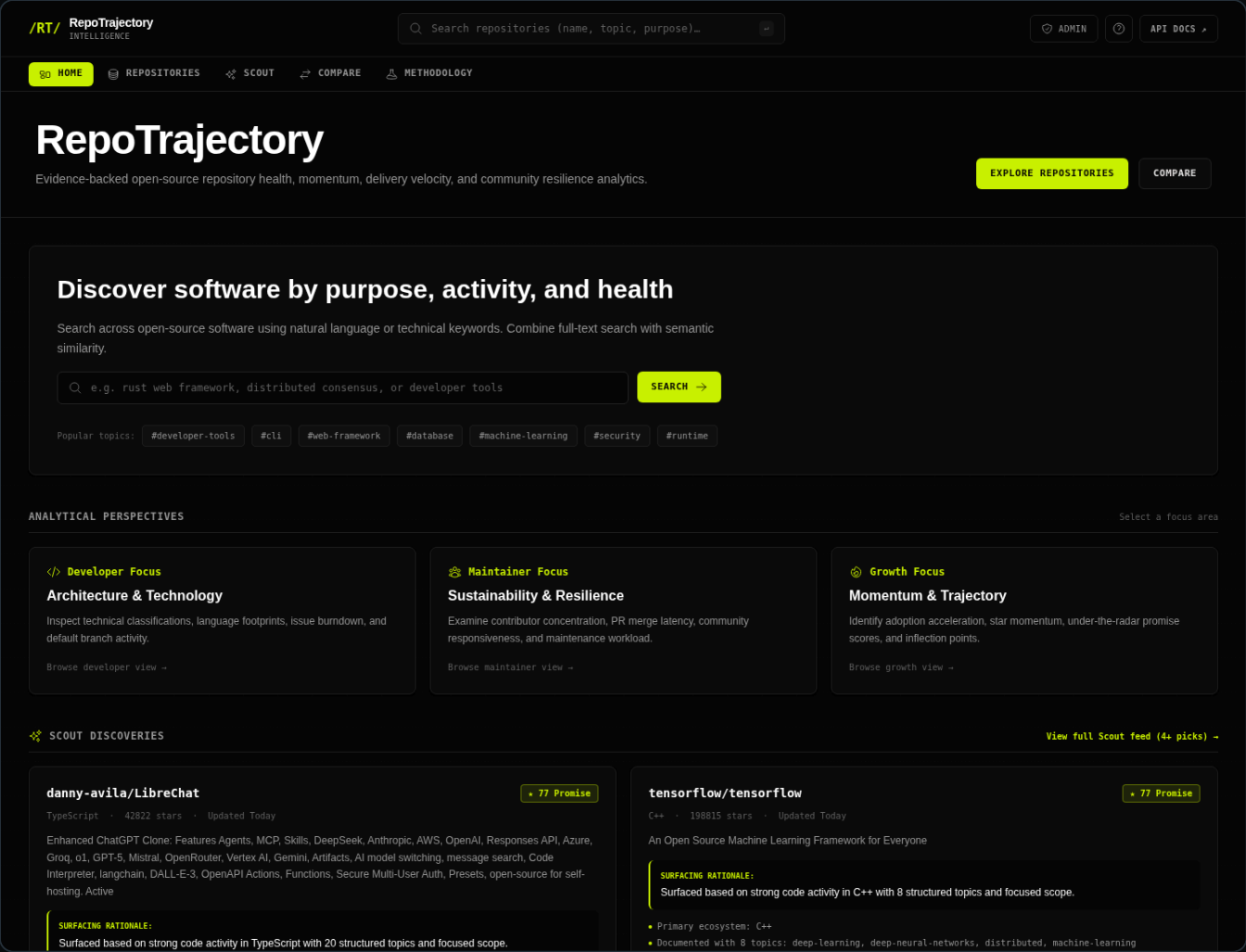
Shared behavior

`AppNavigation` provides global search, links, API docs, admin, and product tour. `FirstRunTutorial` stores tutorial state in browser storage. `MotionShell` animates route entry. Tailwind defines a black/white chartreuse visual language; semantic status still needs text, not color alone.

Data layer

`frontend/lib/api.ts` is the central contract. Pages mix server fetching with client-side re-fetching. Errors generally degrade to empty/null presentation; changing this behavior should include explicit user-facing failure states.

Home and discovery entry point



Live capture, 1440×1100. The page leads with natural-language search, then developer/maintainer/growth lenses and Scout discoveries. Global navigation exposes the public research paths and separates admin access.

Canonical repository directory

/RT/RepoTrajectory

INTELLIGENCE

Q Search repositories (name, topic, purpose)...

ADMIN

API DOCS

HOME

REPOSITORIES

SCOUT

COMPARE

METHODOLOGY

Repositories

Directory of active open-source software repositories across ecosystems, indexed by technology, activity, and health.

+ ADD REPOSITORY

Q Search repositories by name, owner, or topic...

All Languages

DEVELOPER

MAINTAINER

INVESTOR

Stars (Highest)

1,496 repositories in index

Showing 50 items

REPOSITORY	LANGUAGE	STARS	FORKS	PROMISE SCORE	CLASSIFICATION	LICENSE	ACTIVITY
01 openclaw/openclaw DEEP ANALYSIS The AI that really does things. Any OS. Any Platform. The lobster ...	TypeScript	388.8K	81.7K	77 / 100	software	NOASSERTION	Today
02 obra/superpowers DEEP ANALYSIS An agentic skills framework & software development methodology ...	Shell	281.7K	25.2K	—	software	MIT	Yesterday
03 mattpocock/skills DEEP ANALYSIS Skills for Real Engineers. Straight from my .agents directory.	Shell	249.5K	21.1K	—	software	MIT	Today
04 react/react DEEP ANALYSIS The library for web and native user interfaces.	JavaScript	249.1K	51.3K	—	library	MIT	Today
05 affaan-m/ECC DEEP ANALYSIS The agent harness performance optimization system. Skills, instin...	JavaScript	248K	37.4K	—	developer_tool	MIT	Yesterday
06 torvalds/linux Linux kernel source tree	C	244.9K	64.2K	—	software	None	6 days ago
07 NousResearch/hermes-agent DEEP ANALYSIS The agent that grows with you	Python	241.3K	49.5K	—	software	MIT	Today
08 deepseek-ai/deepseek-harness DeepSeek Harness: Everything is a Plugin.	TypeScript	212K	24.9K	—	software	None	Today
09 vuejs/vue DEEP ANALYSIS This is the repo for Vue 2. For Vue 3, go to https://github.com/vuej...	TypeScript	211.6K	33.7K	—	framework	MIT	1 years ago
10 anomalyco/opencode DEEP ANALYSIS The open source coding agent.	TypeScript	203.9K	26.6K	70.8 / 100	software	MIT	Today

Live capture. RepositoryDirectory manages query, language/license/category/evidence filters, lens, sorting, pagination, and expandable repository evidence. Results come from v2 catalog endpoints.

Scout discovery feed

/RT/ RepoTrajectory INTELLIGENCE

Q Search repositories (name, topic, purpose)...

ADMIN

API DOCS

HOME

REPOSITORIES

SCOUT

COMPARE

METHODOLOGY

Scout Feed

Identification of promising repositories based on activity velocity, commit cadence, and maintainer engagement.

98 Evaluated Candidates

All Languages

≥ 60 Standard Tier

Showing 50 discoveries

danny-avila/LibreChat

TypeScript

MIT

Updated Today

Enhanced ChatGPT Clone: Features Agents, MCP, Skills, DeepSeek, Anthropic, AWS, OpenAI, Responses API, Azure, Groq, o1, GPT-5, Mistral, OpenRouter, Vertex AI, Gemini, Artifacts, AI model switching, message search, Code Interpreter, langchain, DALL-E-3, OpenAPI Actions, Functions, Secure Multi-User Auth, Presets, open-source for self-hosting. Active

EVALUATION CONTEXT

Surfaced based on strong code activity in TypeScript with 20 structured topics and focused scope.

SUPPORTING EVIDENCE

Primary ecosystem: TypeScript

Public engagement: 42813 stars, 8863 forks

No elevated risk flags observed

Documented with 20 topics: ai, anthropic, artifacts, aws

Standard open source license: MIT

PROMISE SCORE

77 /100

EMERGING SIGNAL

Quantitative: -

AI Assessment: -

Confidence: 80%

STARS

42.8K

FORKS

8.9K

INSPECT REPOSITORY →

tensorflow/tensorflow

C++

Apache-2.0

Updated Today

An Open Source Machine Learning Framework for Everyone

EVALUATION CONTEXT

Surfaced based on strong code activity in C++ with 8 structured topics and focused scope.

SUPPORTING EVIDENCE

Primary ecosystem: C++

Public engagement: 198807 stars, 76238 forks

No elevated risk flags observed

Documented with 8 topics: deep-learning, deep-neural-networks, distributed, machine-learning

Standard open source license: Apache-2.0

PROMISE SCORE

77 /100

EMERGING SIGNAL

Quantitative: -

AI Assessment: -

Confidence: 80%

STARS

198.8K

FORKS

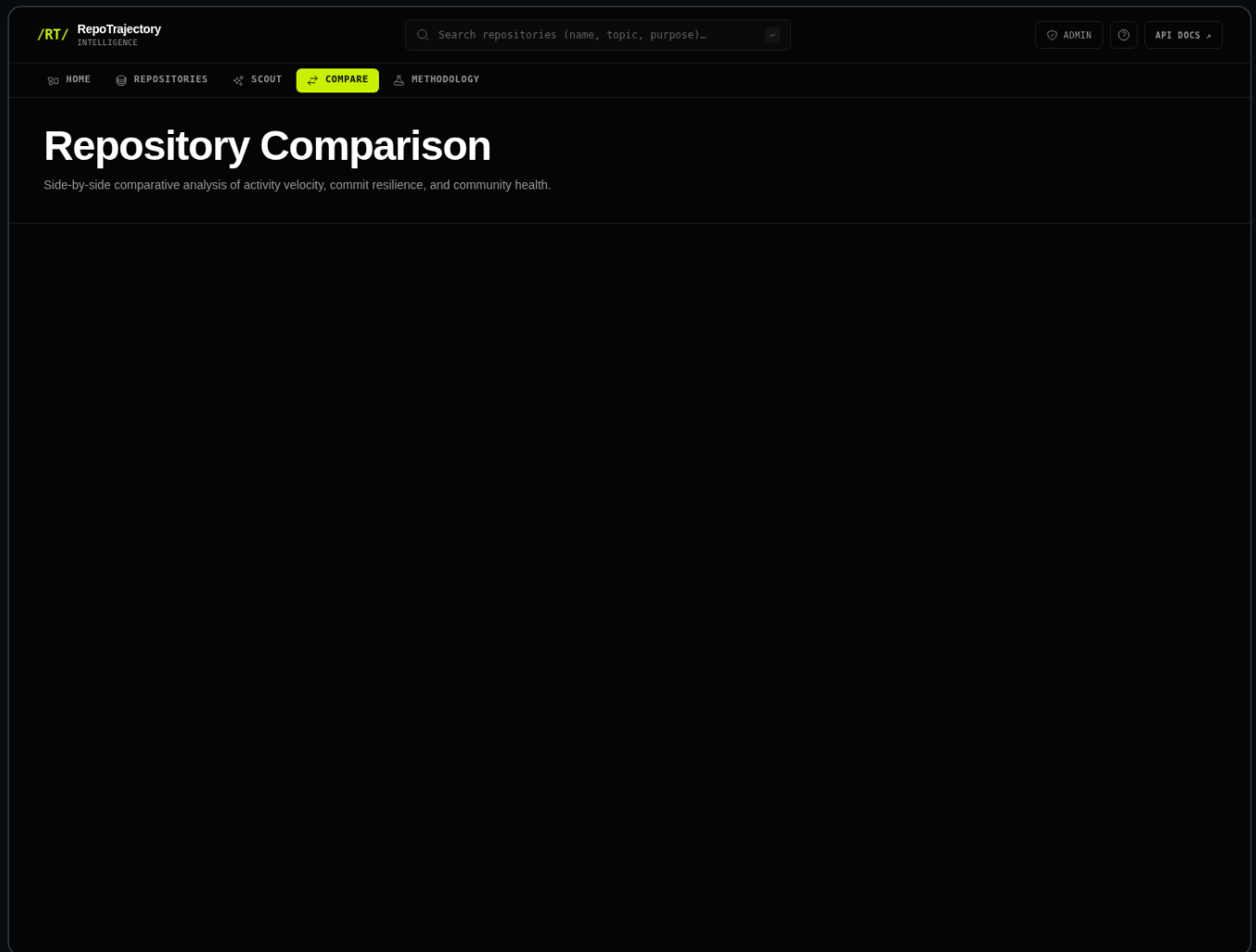
76.2K

INSPECT REPOSITORY →

Live capture. Cards surface promise score, confidence, rationale, supporting facts, uncertainty, and risks. The feed is a decision aid; the explanation payload is as important as rank.

RepoTrajectory technical reference • 41 / 59

Repository comparison



Live capture. **CompareExplorer** lets the user choose repositories and aligns metadata and analytical signals. Empty state is intentionally useful rather than preselecting a potentially misleading comparison.

Unified repository dossier

RT/

RepoTrajectory

INTELLIGENCE

ADMIN

300

900

1Y

API DOCS

HOME

REPOSITORIES

SCOUT

COMPARE

METHODOLOGY

Back to Repositories

300

900

1Y

fastapi/fastapi

DEEP ANALYSIS

STARS

102.1K

FORKS

9.8K

OPEN ITEMS

81

FastAPI framework, high performance, easy to learn, fast to code, ready for production

Python

MIT

Default: master

GitHub

1. Overview & Project Purpose

Project taxonomy, domain classification, and verified purpose excerpt.

PROJECT PURPOSE & DOCUMENTATION SUMMARY

FastAPI framework, high performance, easy to learn, fast to code, ready for production

CLASSIFICATION: FRAMEWORK (94% confidence)

#api

#async

#asyncio

#fastapi

#framework

#json

#json-schema

#openapi

#openapi3

#pydantic

#python

#python-types

#python3

#redoc

#rest

#starlette

#swagger

#swagger-ui

#uvicorn

#web

TELEMETRY COVERAGE

Deep Analysis

Full telemetry: commits, pull requests, cycle times, issue flows, and contributor distributions.

LAST PUSH / FRESHNESS

3 days ago

LICENSE & BRANCH

MIT · Default branch: master

2. Activity & Adoption

Development velocity, commit cadence, star trajectory, and fork-to-star ratio.

Development Activity Over Time (30-Day Window)

Date	Human commits	Merged PRs	Issues closed
Jun 10	45	10	2
Jun 24	48	8	1
Jul 8	22	10	2
Jul 22	48	10	2
Aug 5	12	8	2
Aug 19	13	10	2
Aug 26	2	2	1

Observed Star Growth Trajectory

Date	Observed stars
Jun 10	100000
Jun 24	105000
Jul 8	110000
Jul 22	115000
Aug 5	120000
Aug 19	125000
Aug 26	130000

Methodology governance

/RT/RepoTrajectory

INTELLIGENCE

Q

Search repositories (name, topic, purpose)...

🔍

🔑ADMIN

🔄

API DOCS

🏠HOME

📦REPOSITORIES

🔍SCOUT

⚖️COMPARE

METHODOLOGY

Methodology & Governance

RepoTrajectory makes all metric definitions, formulas, and data boundaries explicit. Scores are deterministic heuristics, AI evaluations run under factual guardrails, and missing evidence is transparently reported.

📦REPOSITORY DIRECTORY

Curated from a 50,000 rolling candidate pool. Max 25% language diversity cap prevents any single ecosystem from dominating the catalog. 500 repos receive deep weekly telemetry.

🔍AI SCOUT ENGINE

70% quantitative pre-ranking + 30% structured AI evaluation. Discovers high-promise projects regardless of star scale (low-star repos like 6-star projects supported).

⚡HYBRID SEARCH (RRF K=60)

PostgreSQL trigram & fulltext lexical search fused with 1536-dimensional pgvector cosine similarity via Reciprocal-Rank Fusion.

Core Questions & Capabilities

Operational coverage across repository intelligence, search, and early detection.

01	How is the canonical 10,000 repository directory selected?	🟢 Available	Selected from a rolling candidate pool of up to 50,000 repositories using composite activity, star growth, release recency, and an enforced 25% maximum language diversity cap to ensure representation across all ecosystems.
02	How does the AI Scout discover under-the-radar projects?	🟢 Available	Autonomous discovery engine combining 70% quantitative signals (velocity, commit cadence, release frequency, issue resolution) with 30% structured AI evaluation. Supports low-star projects (e.g., 6 stars) while strictly filtering out forks, archived repos, and mirrors.
03	How is hybrid search performed across 10,000 repositories?	🟢 Available	Queries execute in parallel across PostgreSQL trigram/fulltext indices and 1536-dimensional pgvector cosine embeddings. Results are merged via Reciprocal-Rank Fusion (RRF, k=60) with automatic deterministic fallback if the vector engine is offline.
04	Which projects are gaining momentum?	🟡 Available with baseline	Evaluates star and contributor growth plus human commit, PR, and release acceleration. Growth remains provisional until snapshots span the window.
05	Which communities are healthy and active?	🟢 Available	Combines active human contributors, delivery cadence, PR merge latency, resolved-item cycle times, and stable release cadence.
06	Which projects depend on very few contributors?	🟡 Available as proxy	Reports top-one and top-three human commit share and Herfindahl-Hirschman Index (HHI). Known bot accounts and automated synchronizers are excluded.
07	How do the Developer, Maintainer, and Investor	🟢 Available	Developer lens prioritizes activity, dependencies, and documentation; Maintainer lens

Live capture. This page makes scoring weights, evidence assumptions, caveats, and status semantics accessible to users. Any formula change should update pure metric code, tests, this page, and docs/metrics.md together.

Collection observability

/RT/ RepoTrajectory INTELLIGENCE

Q Search repositories (name, topic, purpose)...

ADMIN

API DOCS

HOME REPOSITORIES SCOUT COMPARE METHODOLOGY

Collection control

A durable intake pipeline for discovering, qualifying, and refreshing the open-source universe without storing the public event firehose.

REFRESHADMINISTRATION

CANDIDATE UNIVERSE
37,750
35,844 awaiting metadata

ACTIVE COHORT
250
Target 250

FULLY HYDRATED
796
Historical entities stored

QUEUE DEPTH
0
Queue clear

PUBLIC EVENTS SCANNED
9.7M
119 archive hours

DATABASE FOOTPRINT
600.8 MB
59,500 compact event rows

Collection runway

The inexpensive filters run before REST hydration; arrows show the direction of evidence, not a batch dependency.

QUEUE CLEAR

01

Discover

9.7M

GH Archive events streamed and folded; GitHub Search adds established software.

→

02

Qualify

1,807

99 resources and templates withheld.

→

03

Rank

250

Seven-day adoption and collaboration percentiles define the active cohort.

→

04

Hydrate

796

Bounded history, daily snapshots, and incremental refreshes enter PostgreSQL.

Active discovery cohort

Current promotion order. A provisional label means the seven-day GH Archive baseline is still accumulating.

All softwareLibrariesDev toolsHydrated

REPOSITORY	CLASS	7D SIGNALS	DISCOVERY SCORE	STATE
NousResearch/hermes-agent The agent that grows with you	Software	568 adoption + collaboration	88	HYDRATED
openclaw/openclaw The AI that really does things. Any OS. Any Platform. The...	Software	1,642 adoption + collaboration	86	HYDRATED
anomalyco/opencode The open source coding agent.	Software	180 adoption + collaboration	85	HYDRATED

Policy envelope

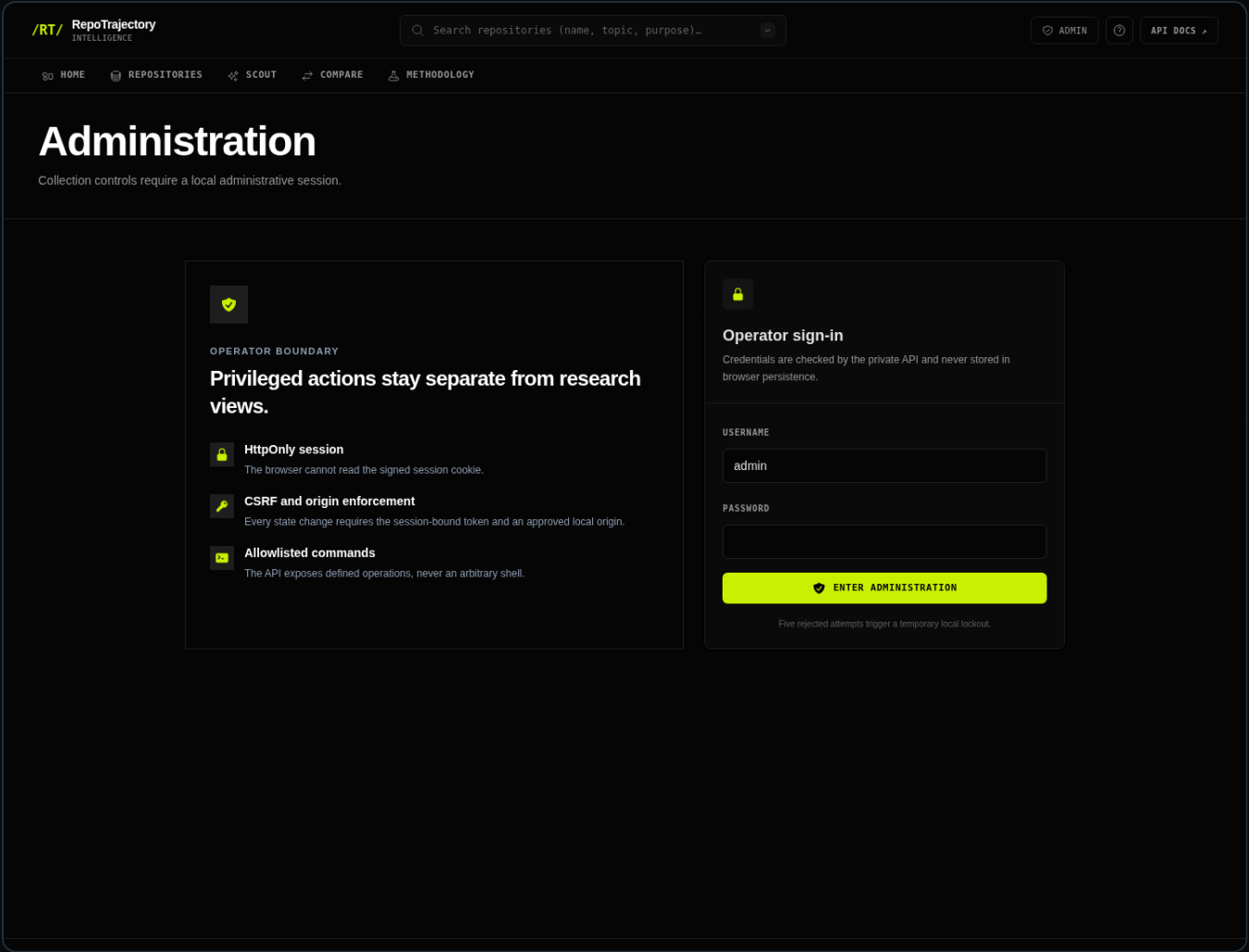
Configured limits make collection cost predictable.

Active set	250
Candidate ceiling	2,000
Refresh cadence	24 hours
Archive projection	119 hours · 1.91 GB read
Git-Hub core budget	4,821 remaining
Last completed work	Today

Live capture. The public collection screen is read-only and summarizes tier/classification distribution, durable jobs, archive processing, rate budget, database footprint, and latest completion.

RepoTrajectory technical reference • 45 / 59

Privileged administration



Live capture of the unauthenticated boundary. Credentials are never embedded in this document. After login, the console offers allowlisted queue/maintenance controls and audit inspection.

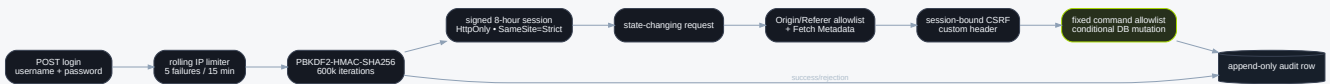
Security boundary and threat controls

Threat	Control
Credential theft from browser storage	Eight-hour HMAC-signed session in HttpOnly, SameSite=Strict cookie; password/session/CSRF never in local storage.
Password disclosure	PBKDF2-HMAC-SHA256, 600,000 iterations, random 128-bit salt; setup writes only hash.
CSRF / cross-site mutation	Session-bound random CSRF header, exact Origin/Referer allowlist, Fetch Metadata rejection.
Brute force	Five rejected logins per observed client address trigger 15-minute rolling logout.
Privilege escalation	Fixed command allowlist; no shell, SQL, Docker socket, or arbitrary job submission.
Race with worker	Status-conditional cancel/retry.
Secret leakage in audit	Structured audit excludes passwords, tokens, CSRF, GitHub/AI keys, signing secret.
Clickjacking/sniffing	DENY frame, nosniff, no-referrer, permissions restrictions, CSP.

Deployment boundary: local defaults use loopback HTTP and `ADMIN_SECURE_COOKIES=false`. For HTTPS exposure, set secure cookies and exact HTTPS allowed origins; replace default database credentials and conduct a review of legacy v1 mutation routes.

Use the least-privileged read-only GitHub token. Public collection needs GET access only; never grant repository deletion, hooks, workflow, packages, organization administration, or broad write scopes.

Authentication and CSRF flow



Token payload

Session contains username, issue/expiry times, and CSRF value. It is base64-url encoded and HMAC authenticated. Decode verifies structure, signature with constant-time comparison, username, and expiration.

Rotation

`./run.sh admin-password` prompts without echo, hashes the new password, generates a new session-signing secret, writes `.env` mode 0600, and invalidates all earlier sessions.

Cookie settings: HttpOnly, SameSite=Strict, configured Secure flag, bounded Max-Age. Admin responses receive `Cache-Control: no-store`.

Core, GitHub, and collection settings

Environment / field	Default	Meaning
DATABASE_URL	localhost:15432 dev URL	Async PostgreSQL connection; Compose overrides host.
GITHUB_TOKEN	none	Read-only token; needed for practical quota.
GITHUB_API_URL	api.github.com	REST base; injectable for tests.
GITHUB_REQUEST_INTERVAL_SECONDS	0.15	Serialized minimum spacing per client.
GITHUB_RATE_LIMIT_RESERVE	100	Preserved budget.
INGESTION_BOOTSTRAP_DAYS	180	Initial commit horizon.
INGESTION_RELEASE_DAYS	730	Release horizon.
INGESTION_*_LIMIT	200 / 5000 / 3000 / 3000 / 500	Contributors, commits, issues, PRs, releases.
COLLECTOR_POLL_SECONDS	10	Idle loop interval.
COLLECTOR_LEASE_MINUTES	30	Worker-death recovery threshold.
COLLECTOR_CANDIDATE_LIMIT	2000	Legacy policy target/ceiling.
COLLECTOR_ACTIVE_LIMIT	250	Legacy recurring hydration cohort.
COLLECTOR_ACTIVE_REFRESH_HOURS	24	Recurring deep refresh cadence.
COLLECTOR_CANDIDATE_REFRESH_HOURS	168	Candidate metadata cadence.

Discovery, AI, search, and capacity settings

Environment / field	Default	Operational impact
DISCOVERY_LANGUAGES	Python, TS, JS, Go, Rust, Java	Legacy queries and scheduled canonical shards.
DISCOVERY_MIN_STARS	500	Legacy Search intake only.
DISCOVERY_PUSHED_WITHIN_DAYS	180	Freshness eligibility.
GH_ARCHIVE_HOURS_BACK / LAG_HOURS	6 / 3	Hourly replay window and source delay.
GH_ARCHIVE_TOP_REPOSITORIES_PER_HOUR	500	Projection bound.
GH_ARCHIVE_RETENTION_DAYS	90	Activity/file ledger retention.
AI_BASE_URL	Gemini OpenAI-compatible URL	Provider endpoint.
GEMINI_API_KEY / AI_API_KEY	none	Hosted provider; Gemini takes precedence.
AI_EVALUATION_MODEL	gemini-3.8-flash	Scout structured judgment.
AI_EMBEDDING_MODEL	text-embedding-3-small	Semantic search model label.
DIRECTORY_LIMIT / DEEP_COHORT_LIMIT	10000 / 500	Canonical membership targets.
CANDIDATE_POOL_LIMIT / RETENTION_DAYS	50000 / 90	Broad-pool bounds.
DIRECTORY_LANGUAGE_CAP	0.25	Maximum initial share per language.
SCOUT_DAILY_EVAL_LIMIT	100	Daily assessment workload.
SEARCH_RRF_K / SEARCH_DEFAULT_LIMIT	60 / 50	Fusion damping and response size.

The checked-in `.env.example` is the operator template. This book intentionally omits runtime credential values.

Local development workflow

Recommended full stack

```
cp .env.example .env
# set a read-only GITHUB_TOKEN
./run.sh

# lifecycle
./run.sh logs
./run.sh down
./run.sh admin-password
```

`run.sh` checks host ports 10100–10110, remembers/reuses the current app port, builds/starts Compose, waits for readiness, and prints UI, collection, admin, and API-doc links.

Split-process backend development

```
docker compose -f docker-compose.yml -f docker-compose.dev.yml up -d postgres
cd backend
python -m venv .venv && source .venv/bin/activate
pip install -e '.[dev]'
alembic upgrade head
uvicorn app.main:app --reload --port 8001

# second shell
python -m app.cli collector
```

Frontend

```
cd frontend
npm install
npm run dev
```

The development proxy sends `/backend` to `localhost:8001`. PostgreSQL is exposed only by the dev override on host port 15432.

Before coding: check `git status --short`, preserve unrelated user changes, run the smallest relevant tests, and use Alembic for schema changes.

Docker, startup, and proxy behavior

Startup order

1. PostgreSQL and Redis reach health.
2. API runs `alembic upgrade head`, then Uvicorn.
3. Collector waits for healthy API.
4. Web depends on API health.
5. Caddy routes after web/API exist.

Network segmentation

All services join `internal`; application services needing outbound access also join `public`. Only proxy publishes a host port. Database and API use Compose `expose`, not host `ports`.

Caddy route contract

```
:80 {  
  handle_path /backend/* { reverse_proxy api:8000 }  
  handle { reverse_proxy web:3000 }  
}
```

Persistence

`postgres_data` holds the database, `backups_data` holds backup files, `redis_data` holds Redis state, and Caddy has data/config volumes. Stopping containers is recoverable; deleting volumes is destructive.

Production hardening: local database credentials are embedded in Compose for convenience. A public deployment needs external secret management, TLS, exact origins, secure cookies, monitoring, backup scheduling, and explicit network/firewall policy.

CLI, observability, and routine operations

Command	Use
<code>python -m app.cli schedule</code>	Run one scheduler tick.
<code>... collector [--once]</code>	Continuous worker or one job.
<code>... collector-status</code>	Queue, archive, DB, rate overview.
<code>... enqueue owner/repo</code>	Pin and prioritize one repository.
<code>... ingest owner/repo</code>	Immediate one-off ingestion.
<code>... ingest-config ../repositories.yml</code>	Ingest configured list.
<code>... classify-candidates</code>	Reapply transparent eligibility.
<code>... reconcile-directory</code>	Rebuild canonical memberships.
<code>... scout-eval --limit N</code>	Run Scout assessments.
<code>... generate-embeddings --limit N</code>	Populate catalog vectors.
<code>... discover-sharded ...</code>	Run explicit catalog shard.
<code>... seed-benchmark / benchmark-latencies</code>	Create/measure scale scenarios.

Watch first

Oldest queued age, failed jobs and errors, latest completion, GitHub remaining/reset, last GH Archive hour and failure, database size/write latency, directory/deep counts, API latency, AI degradation, snapshot span, and evidence sample sizes.

Normal recovery: restart the collector. Leases expire, jobs requeue, upserts tolerate replay, and projections replace atomically.

Backup, restore, and disaster recovery

Backup script

`scripts/backup.sh` produces timestamped compressed custom-format `pg_dump` files in `/backups/daily`. Sundays are copied to weekly.

- Daily retention: 7 days.
- Weekly retention: 28 days.
- Defaults target Compose PostgreSQL.

Restore script

`scripts/restore.sh` accepts a backup and optional `--rehearsal`. Rehearsal creates an isolated database, restores, checks key table counts, and drops it.

- `--clean --if-exists`.
- `--no-owner --no-acl`.
- Verifies catalog/repo/embedding/scout counts.

Recovery objectives by data class

Data	Recoverability
GitHub entities	Replayable within current API availability and configured bounds.
GH Archive compact hours	Replayable from public archive; algorithm version controls replacement.
Observed historical snapshots	Not reconstructable exactly after loss; backups matter.
Job/audit history	Operational evidence; not fully recreated from sources.
Embeddings/Scout	Regenerable if provider/config remains available.

Restore-script caveat: `pg_restore ... || true` can mask restore errors. A production recovery procedure should inspect command status/logs and validate more than four counts before declaring success.

Tests and quality gates

Backend test coverage

- Pure metric transforms and missing baselines.
- GitHub retries, rate limits, pagination.
- Discovery aggregation/classification.
- Directory selection/diversity.
- Hybrid search fusion, fallback, cache.
- Scout scoring and guardrails.
- v2 list/profile/scout/facets/health.
- Admin password/session/origin behavior.
- Migration 0007 expectations.

Primary gates

```
cd backend
PYTEST_DISABLE_PLUGIN_AUTOLOAD=1 \
  pytest -p pytest_asyncio.plugin
ruff check app tests alembic/versions
mypy app

cd ../frontend
npm run build

cd ..
docker compose config --quiet
```

Test architecture

Fixtures use async SQLite for fast service/API isolation and dependency override. PostgreSQL-specific behavior—pgvector, SKIP LOCKED, dialect inserts, indexes, and migration execution—requires integration validation against the real Compose database.

Generation-time verification

Gate	Observed result on 2026-09-05
Frontend production build	Pass; Next.js compiled, type-checked, and generated all routes.
Compose configuration	Pass.
Focused admin-auth tests	3 passed.
Full backend pytest	Incomplete; advanced four tests, then exceeded a 120-second diagnostic timeout.
Ruff / strict MyPy	Failing baseline: 120 lint findings; 9 type errors in 4 source files.

Current gap: there is no dedicated frontend unit/e2e suite in the repository. Build catches TypeScript/render failures, while screenshots and manual browser checks cover workflows. Add focused interaction tests before high-risk UI refactors.

Live local stack snapshot

Observed from the running Compose stack. These values demonstrate scale and health; they are not configuration constants.

CATALOG CANDIDATES

33,940

DIRECTORY

1,496

DEEP COHORT

595

HYDRATED REPOS

796

Signal	Observed value
v2 health	Degraded only because hosted AI credentials were not configured; fallback heuristic active; database connected.
Legacy tiers	250 active, 37,499 candidate, 1 pinned.
Jobs	6,005 completed; 467 cancelled; 4 failed; no queued oldest timestamp at observation.
GH Archive	119 hours; 9,712,772 events; 2,046,718,076 compressed bytes; 59,500 retained activity rows.
Database size	629,963,799 bytes (~601 MiB).
GitHub budget snapshot	Limit 5,000; remaining 4,821; used 179 in reported window; process request counter 1,180.
Collection	oss-libraries, enabled; policy 2,000 candidates / 250 active / 24-hour refresh; observed membership count exceeded nominal candidate_limit.

Model reconciliation: v2 deep count (595), configured target (500), and legacy hydrated count (796) differ because they measure different tables/flags and current state can drift between reconciliations. Do not compare them as identical KPIs.

Limits, ambiguity, and technical debt

Risk / limitation	Impact and mitigation
Historical growth begins at first observation	Early momentum is provisional; expose history availability and snapshot span.
Default-branch commits only	Activity can miss branch/monorepo workflows; describe scope.
Identity and bot heuristics	Contributor breadth/automation can be wrong; keep raw components and basis.
Cycle-time semantics	Not response/review time; avoid stronger labels.
Dual legacy/canonical models	Counts and membership can diverge; define source-of-truth per feature.
Redis provisioned but unused	Operational complexity without current application value; document or remove deliberately.
Legacy v1 mutation routes	Potential security inconsistency; require explicit protection audit before exposure.
Offset cursor	Mutation can reorder pages; consider keyset pagination.
Fixed vector dimension	Model changes need migration/re-embedding.
Frontend lacks test suite	Interaction regressions rely on build/manual checks.
Restore masks pg_restore exit	False recovery success; tighten failure handling and validation.
External source quotas/outages	Backlog and stale evidence; preserve durable retries, degraded flags, freshness.

Product principle: uncertainty should be displayed, not “fixed” by inventing neutral evidence or silently substituting stale values.

Safe extension playbooks

Add a collected GitHub field

Update ORM + Alembic, GitHub payload mapping/upsert, Pydantic schema, catalog sync if public, TypeScript type/UI, tests, docs. Decide null and historical semantics.

Add a metric

Implement pure transform, build query inputs, version component JSON, store samples/coverage, expose API, document methodology/caveats, test zero/missing/extremes, update UI.

Add a job type

Define dedupe scope, priority, attempts, payload schema, scheduler/manual producer, worker branch, replay/idempotency, rate-limit behavior, audit/admin exposure, tests and observability.

Add a v2 endpoint

Add response schema, route with session dependency, service query, bounded filters/pagination, API tests, TypeScript contract/helper, loading/empty/error UI.

Change AI model

Check protocol and dimensions, bump embedding version, migrate vector size if needed, invalidate/regenerate embeddings/cache, preserve fallback, record model/prompt identity, latency-test.

Change selection policy

Keep component explanation, test diversity/bounds, reconcile both universes deliberately, preview membership churn, bump projection version if GH Archive fold changes.

Minimum acceptance sequence

```
inspect writers/readers → make narrow change → migrate if needed
→ run focused tests → run full backend gates → frontend build
→ Compose config/health → inspect UI and operational telemetry
```

Never mutate generated/current evidence to hide a model inconsistency. Fix the producing path, version the methodology if meaning changes, and let scheduled reconciliation/refresh repair state.

Invariant checklist and file index

Non-negotiable invariants

- ✓ Use GitHub numeric ID as stable external identity.
- ✓ Never persist raw GH Archive payloads.
- ✓ Every collection dimension stays bounded.
- ✓ Jobs are deduplicated, leased, retryable, and replay-safe.
- ✓ Missing evidence remains explicit.
- ✓ Scores retain components, samples, coverage, version.
- ✓ Hosted AI remains optional.
- ✓ Browser never receives source tokens/secrets.
- ✓ Admin mutations require session + origin + CSRF.
- ✓ Schema changes ship with Alembic.
- ✓ Public API and TypeScript contracts change together.
- ✓ UI labels do not overclaim metric meaning.

First file by question

Config	core/config.py
Schema	db/models.py
Queue	services/collector.py
Discovery	services/discovery.py
Hydration	services/ingestion.py
Metrics	analytics/metrics.py services/analytics.py
Catalog	services/catalog.py services/directory.py
Scout	services/scout.py
Search	services/search.py
HTTP	api/routes.py api/v2/routes.py
Security	core/admin_auth.py api/admin.py
Web client	frontend/lib/api.ts

Final orientation sentence: RepoTrajectory is a bounded, replayable evidence pipeline whose product value comes from preserving provenance and uncertainty while translating broad public repository activity into searchable, explainable, role-specific decisions.

Companion sources: README.md • docs/collector.md • docs/metrics.md • docs/administration.md • OpenAPI at /backend/docs